Database Modeling CheatSheet

{ik} INTERVIEW KICKSTART

Table Of Content

- Introduction to Relational Databases
- ACID Properties
- Differences between SQL & NoSQL
- Vector Databases
- Relational Data Model
- Keys in Relational Databases
- Database Design Process
- Entity-Relationship Diagrams
- Normalization Techniques
- Database Security and Authorization

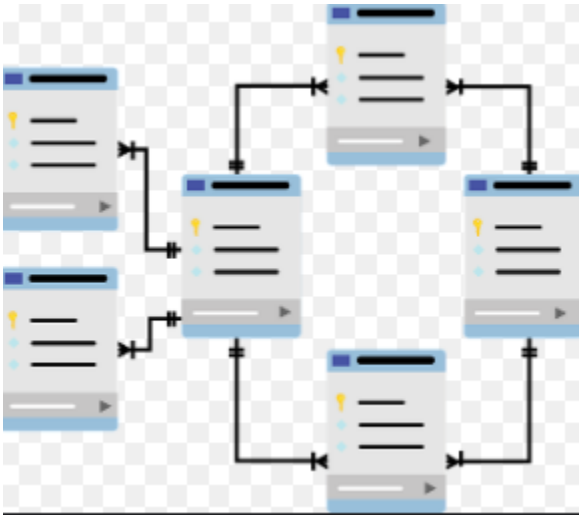
Introduction

Database modeling is a critical aspect of designing and implementing databases that efficiently store, manage, and retrieve data. It involves structuring data according to the needs of a particular application or use case, ensuring data integrity, performance, and scalability. This cheat sheet covers the fundamental concepts of database modeling, including relational databases, ACID properties, normalization, and security. It also touches on emerging topics like vector databases and NoSQL, making it a comprehensive guide for anyone looking to strengthen their understanding of database systems.

What are Relational Databases?

- **Definition:** A database is a structured collection of data that enables efficient storage, retrieval, and management of information.
- **Historical Evolution:** From paper-based records to digital systems, databases have transformed the way data is stored and accessed.
- **Role in Modern Society:** Databases are pivotal in various industries, essential for managing vast amounts of data in today's digital age.

Relational Databases



Introduction: The relational data model organizes data using tables with entities, attributes, and relationships.

Key Principles:

- **Entities:** Objects or concepts (e.g., "Customer" or "Product").
- **Attributes:** Properties of entities (e.g., "Customer Name" or "Product Price").
- **Relationships:** How entities are connected.

Advantages:

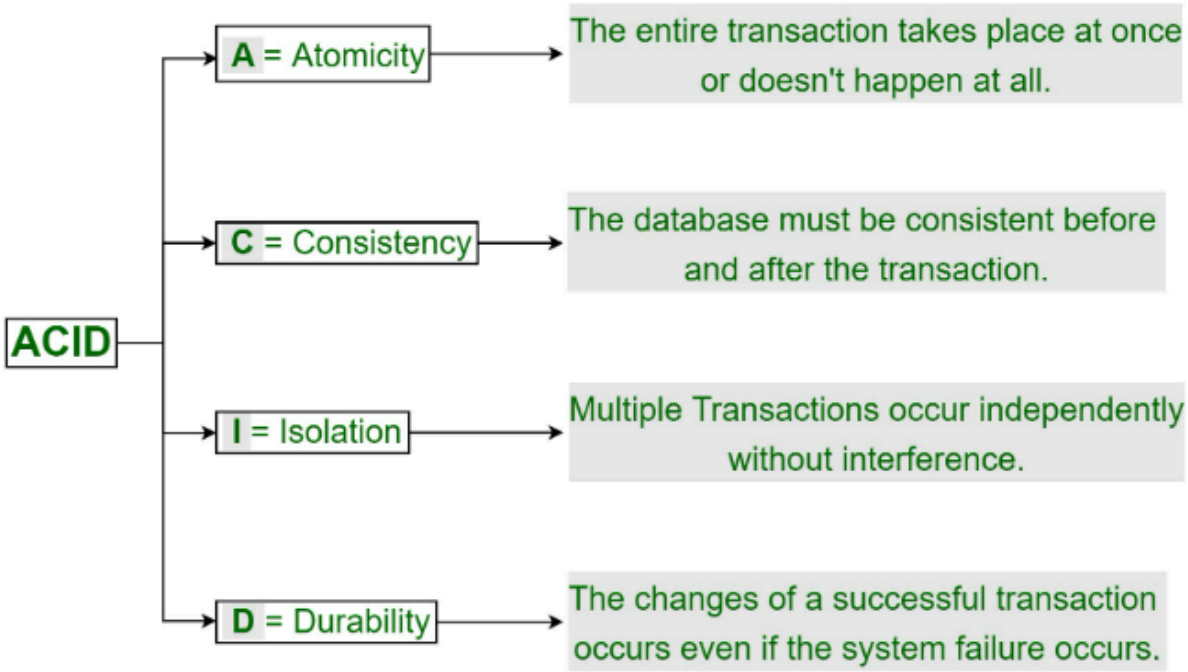
- **Data Integrity:** Ensures accuracy and consistency.
- **Flexibility:** Easily adaptable to changes.
- **Scalability:** Can handle large datasets and complex queries.

ACID Properties

ACID (Atomicity, Consistency, Isolation, Durability) is a set of properties that guarantee the reliability and integrity of transactions in a relational database management system (RDBMS).

These properties help differentiate RDBMS fromNoSQL (Not Only SQL) alternatives, which often prioritize flexibility and scalability over strict ACID compliance.

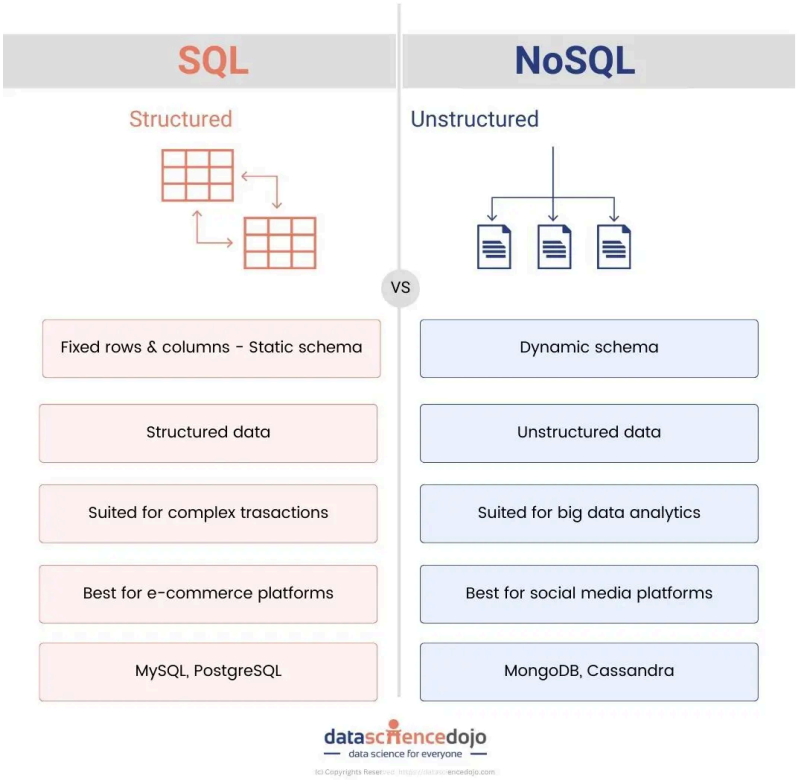
- **Atomicity:** Ensures that transactions are all-or-nothing.
- **Consistency:** Guarantees that data remains consistent after a transaction.
- **Isolation:** Ensures that transactions do not interfere with each other.
- **Durability:** Ensures that once a transaction is committed, it remains so even in case of a failure.



{ik} INTERVIEW KICKSTART

Differences between SQL & No-SQL

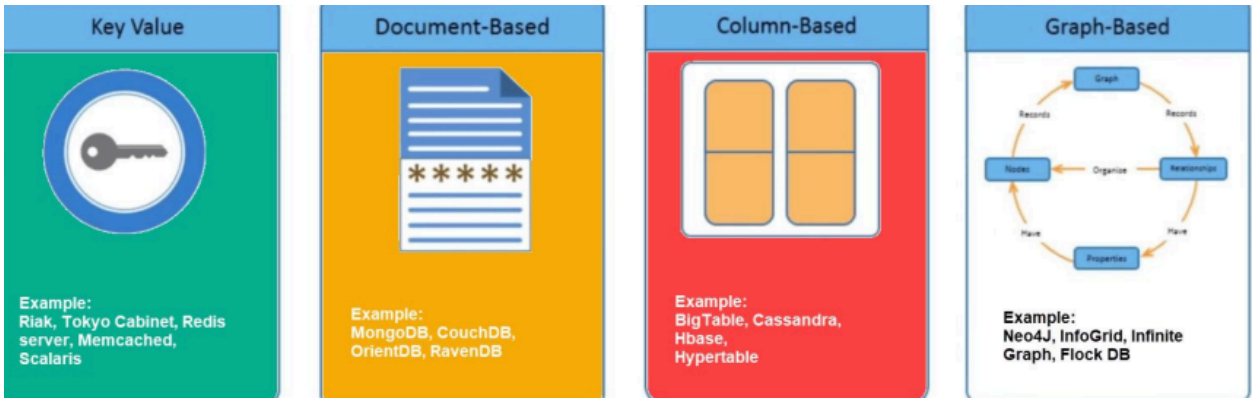
- **SQL:** Structured data with a fixed schema, strong ACID compliance, and support for complex queries and transactions.
- **NoSQL:** Flexible schema, horizontal scalability, often used in large-scale distributed data environments where performance and flexibility are prioritized over consistency.



{ik} INTERVIEW KICKSTART

Why is No-SQL used?

- **Flexible Schema:** NoSQL databases offer schema flexibility, ideal for handling unstructured or evolving data without predefined structures.
- **Scalability:** NoSQL scales horizontally, making it suitable for high data
  volumes and traffic loads, ensuring performance.
- **High Throughput:** NoSQL excels in high read/write throughput scenarios,
  beneficial for real-time analytics and IoT applications.
- **Big Data Support:** NoSQL handles vast data efficiently, making it a top
  choice for big data applications and distributed processing.
- **Data Model Variety:** NoSQL offers diverse data models (document, key-value, column-family, graph) to align with specific application requirements.



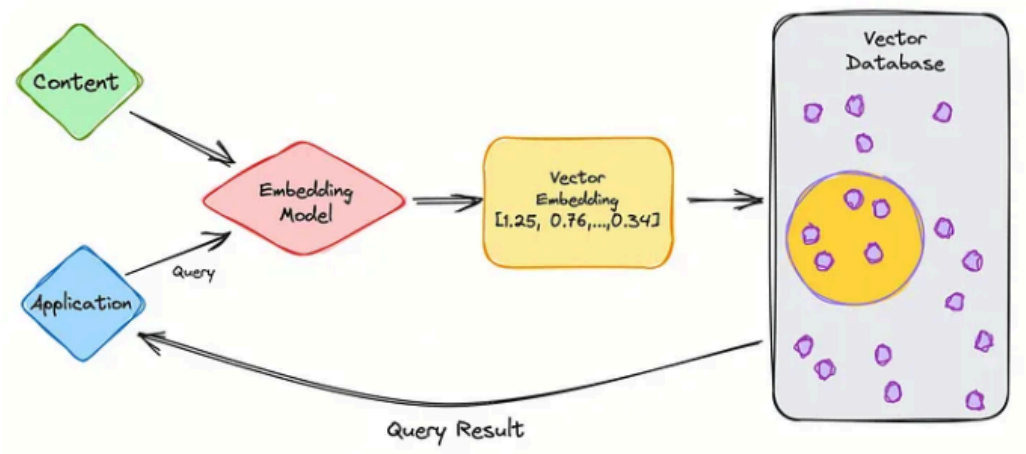
{ik} INTERVIEW KICKSTART

Vector Databases

A vector database stores pieces of information as vectors. Vector databases cluster related items together, enabling similarity searches and the construction of powerful AI models.
Vector embeddings are a way of turning complex objects into numerical vectors that capture their characteristics, and vector databases use these embeddings to efficiently search and retrieve similar or relevant objects based on their positions in the vector space.

Emerging Applications: Crucial for AI systems, large language models, and semantic search due to their ability to handle vector embeddings.

How Do Vector Databases Work?



{ik} INTERVIEW KICKSTART

User Query:

- You input a question or request into the ChatGPT application.

Embedding Creation:

- The application converts your input into a compact numerical form called a vector embedding.
- This embedding captures the essence of your query in a mathematical representation.

Database Comparison:

- The vector embedding is compared with other embeddings stored in the vector database.
- Similarity measures help identify the most related embeddings based on content.

Output Generation:

- The database generates a response composed of embeddings closely matching your query's meaning.

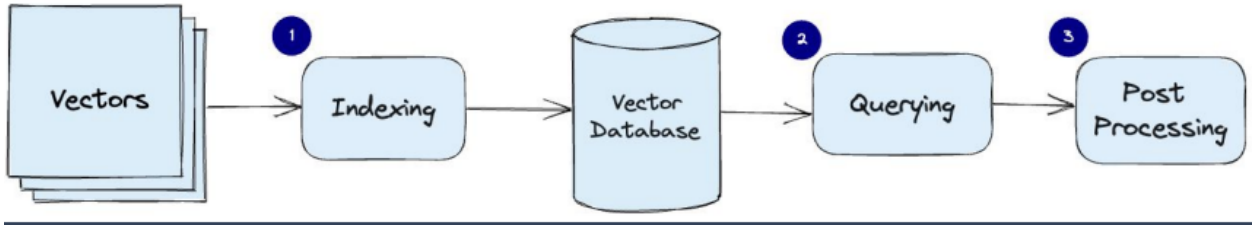
User Response:

- The response, containing relevant information linked to the identified embeddings, is sent back to you.

Follow-up Queries:

- When you make subsequent queries, the embedding model generates new embeddings.
- These new embeddings are used to find similar embeddings in the database, connecting back to the original content.

Here's a common pipeline for a vector database:



How Vector Databases Know Which Vectors are Similar?

A vector database determines the similarity between vectors using various mathematical techniques, with one of the most common methods being **cosine similarity.**

The vector representation of the search query is compared to the vector representations of all the player profiles in the database using cosine similarity. The more similar the vectors are, the higher the cosine similarity score.

Real-World Examples of Different Databases

- **Relational Database as a Service (RDBaaS):** Examples include Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL.
- **NoSQL Databases:** Examples include MongoDB Atlas, Amazon DynamoDB, Google Cloud Firestore.
- **In-Memory Databases:** Examples include Amazon ElastiCache, Azure Cache for Redis.
- **Graph Databases:** Examples include Amazon Neptune, Azure Cosmos DB.

The Relational Data Model

The relational model represents how data is stored in Relational Databases. A relational database consists of a collection of tables, each of which is assigned a unique name.

Entities and Attributes

- **Entities:** Represent real-world objects or concepts (e.g., "Customer" or "Product").
- **Attributes:** Describe the properties of these entities (e.g., "Customer Name" or "Product Price").

Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE shown in the table.

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI		18

Important Terminologies:

- **Attribute:** Attributes are the properties that define an entity. e.g.; ROLL_NO, NAME, ADDRESS

- **Relation Schema:** A relation schema defines the structure of the relation and represents the name of the relation with its attributes. e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation, it is called Relational Schema.

- **Tuple:** Each row in the relation is known as a tuple. The above relation contains 4 tuples, one of which is shown as:

1	RAM	DELHI	9455123451	18

- **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. Table 1 shows the relation instance of STUDENT at a particular time. It can change whenever there is an insertion, deletion, or update in the database.

- **Degree:** The number of attributes in the relation is known as the degree of the relation. The STUDENT relation defined above has degree 5.

- **Cardinality:** The number of tuples in a relation is known as cardinality. The STUDENT relation defined above has cardinality 4.

- **Column:** The column represents the set of values for a particular attribute. The column ROLL_NO is extracted from the relation STUDENT.

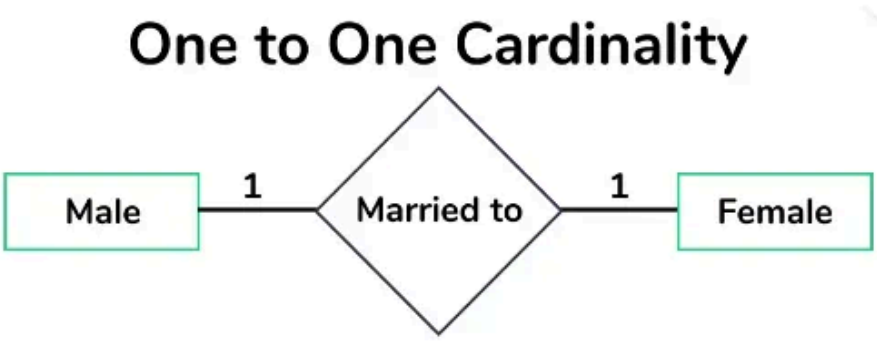
ROLL_NO
1
2
3
4

{ik} INTERVIEW KICKSTART

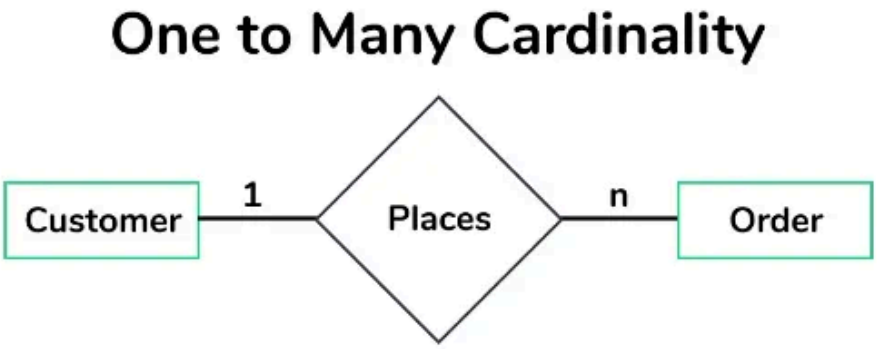
Relationships

How entities are connected in a database (e.g., how students are related to courses they enroll in).

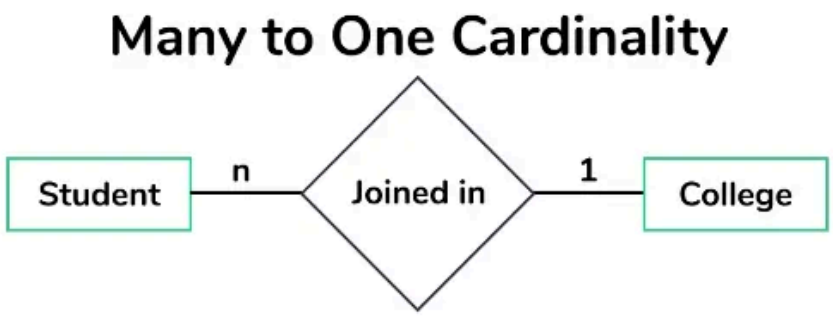
1. **One-to-One Relationships:** One entity is related to one instance of another entity.



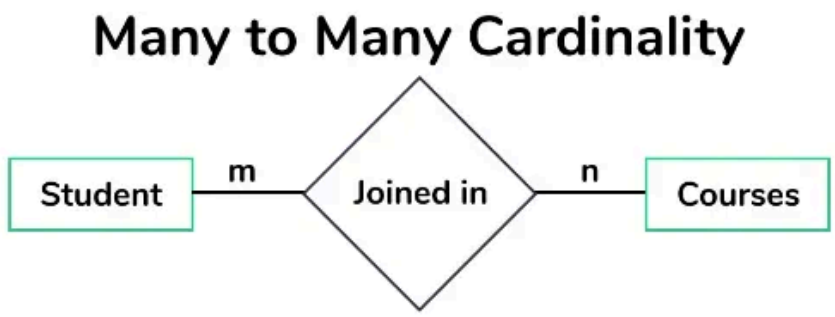
2. **One-to-Many Relationships:** One entity is related to multiple instances of another entity.

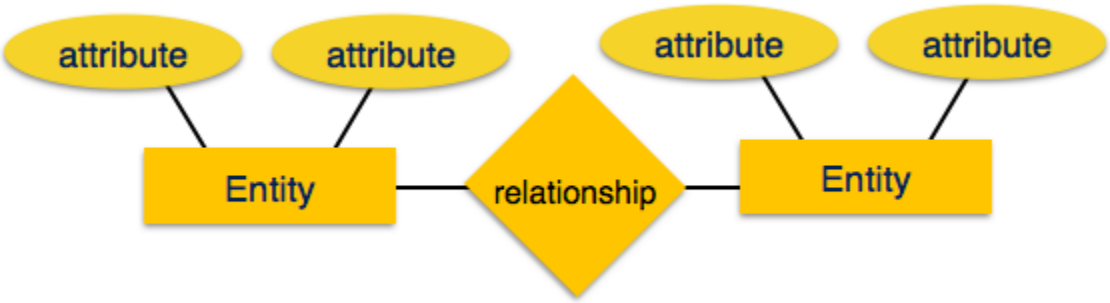


3. **Many-to-One Relationships:** Multiple instances of one entity are related to multiple instances of another entity.



4. **Many-to-Many Relationships:** Multiple instances of many entities are related to multiple instances of another entity.

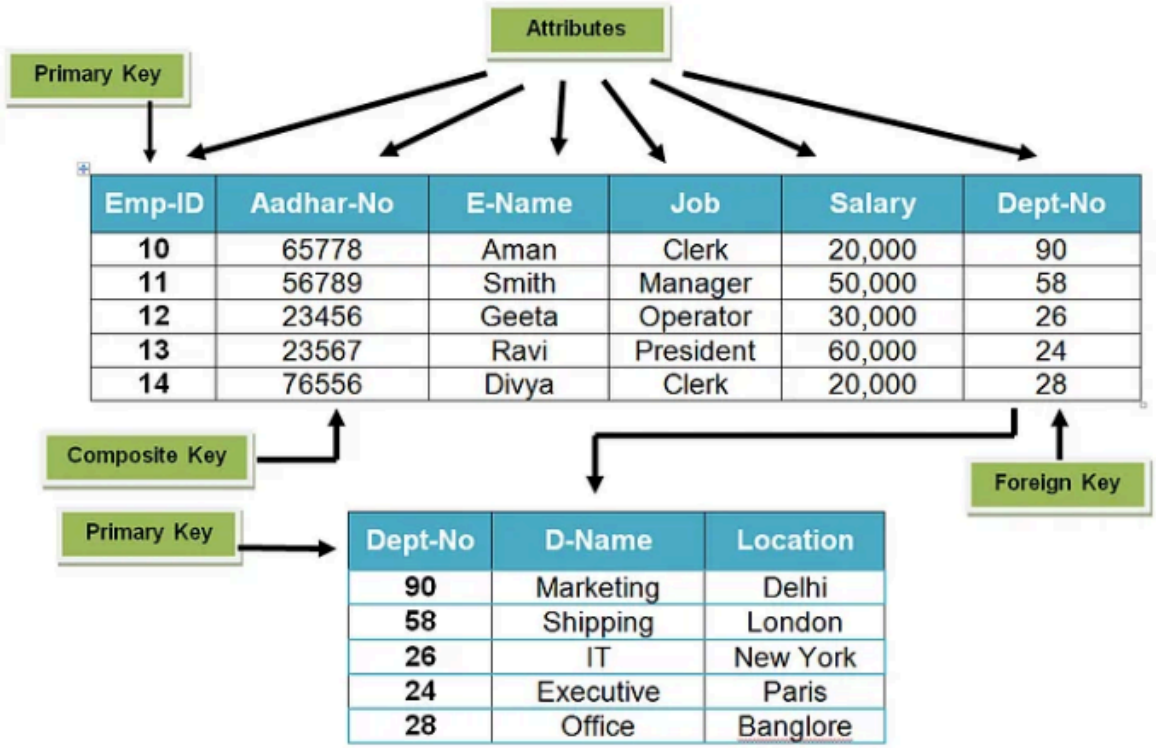




{ik} INTERVIEW KICKSTART

Keys in Relational Databases

- **Primary Keys:** Uniquely identify records within a table.
- **Candidate Keys and Super Keys:** Potential primary keys; combinations of attributes that uniquely identify records.
- **Foreign Keys:** Establish relationships between tables, maintaining referential integrity.



Emp-ID	Aadhar-No	E-Name	Job	Salary	Dept-No
10	65778	Aman	Clerk	20,000	90
11	56789	Smith	Manager	50,000	58
12	23456	Geeta	Operator	30,000	26
13	23567	Ravi	President	60,000	24
14	76556	Divya	Clerk	20,000	28

Dept-No	D-Name	Location
90	Marketing	Delhi
58	Shipping	London
26	IT	New York
24	Executive	Paris
28	Office	Banglore

Example: Defining Keys in a Table

```sql
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    CustomerEmail VARCHAR(100) UNIQUE
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Data Integrity and Constraints

- **NOT NULL:** Ensures that a column cannot have a NULL value.
- **UNIQUE:** Ensures that all values in a column are unique.
- **CHECK:** Ensures that all values in a column satisfy a specific condition.
- **DEFAULT:** Sets a default value for a column when no value is provided.

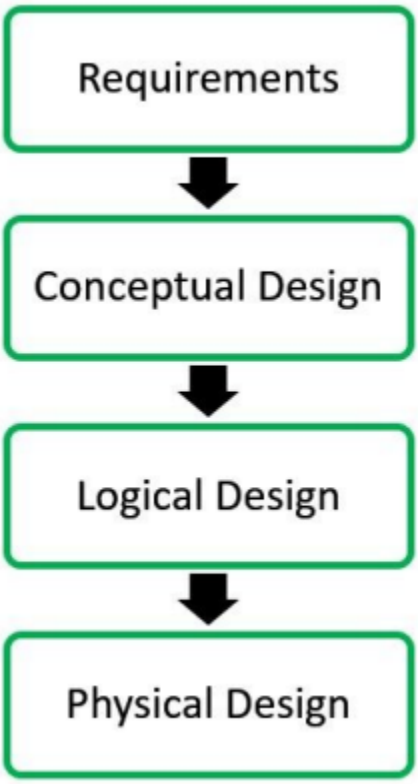
Example: Implementing Constraints

```sql
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) CHECK (Price > 0),
    StockQuantity INT DEFAULT 0
);
```

Database Design Process

Phases of Database Design:

1. **Requirements Gathering:** Understand the needs of the users and the system.
2. **Conceptual Design:** Create an abstract model of the database using entities, attributes, and relationships.
3. **Logical Design:** Refine the conceptual model into detailed tables, relationships, and constraints.
4. **Physical Design:** Implement the database in a DBMS, optimizing for performance and security.



Step 1: Requirements Gathering

- **Customer Needs:** Understand product selection, search capabilities, and ordering options.
- **Inventory Management:** Define how inventory tracking should work.
- **Payment and Checkout:** Specify payment methods and security measures.
- **User Accounts:** Identify required information for customer profiles.
- **Delivery:** Clarify delivery preferences and scheduling options.

Step 2: Conceptual Design

- **Entities:** Identify primary entities like "Products," "Customers," "Orders," and "Inventory."
- **Attributes:** Define attributes for each entity.
- **Relationships:** Establish connections between entities.

Step 3: Logical Design

- **Tables:** Create tables for each entity, specifying fields.
- **Data Types:** Define data types for fields.
- **Constraints:** Implement primary keys, foreign keys, and unique constraints.
- **Normalization:** Optimize the structure, reducing redundancy.

Step 4: Physical Design

- **DBMS Selection:** Choose a suitable DBMS like MySQL, PostgreSQL.
- **Indexing:** Determine which fields require indexing for faster queries.
- **Partitioning:** Consider partitioning strategies for large datasets.
- **Security:** Implement security measures to protect data.
- **Performance Tuning:** Optimize database performance through query optimization.

Example: Translating Logical Design into SQL

```sql
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    CustomerEmail VARCHAR(100),
    CustomerAddress VARCHAR(255)
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    TotalPrice DECIMAL(10, 2),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

Entity-Relationship Diagrams (ERD)

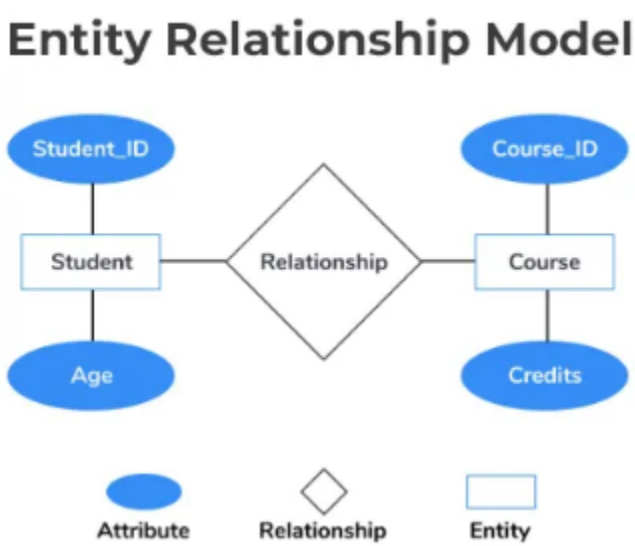
Introduction to ERD

Visual tools that provide a graphical representation of a database's structure, showing entities, attributes, and relationships.

Components of an ERD

- **Entities:** Represent real-world objects or concepts.
- **Attributes:** Properties or characteristics of entities.
- **Relationships:** Establish connections between entities**.**
- **Cardinality:** Defines the number of related records between entities.

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity



Normalization Techniques for Efficient Databases

Normalization is a database design technique that helps us organize data efficiently, reducing redundancy and ensuring data integrity.

Levels of Normalization

- **First Normal Form (1NF):** This is the most basic level of normalization. In 1NF, each table cell should contain only a single value, and each column should have a unique name. The first normal form helps to eliminate duplicate data and simplify queries.

- **Second Normal Form (2NF):** 2NF eliminates redundant data by requiring that each non-key attribute be dependent on the primary key. This means that each column should be directly related to the primary key, and not to other columns.
- **Third Normal Form (3NF):** 3NF builds on 2NF by requiring that all non-key attributes are independent of each other. This means that each column should be directly related to the primary key, and not to any other columns in the same table.
- **Boyce-Codd Normal Form (BCNF):** BCNF is a stricter form of 3NF that ensures that each determinant in a table is a candidate key. In other words, BCNF ensures that each non-key attribute is dependent only on the candidate key.

Normal forms help to reduce data redundancy, increase data consistency, and improve database performance. However, higher levels of normalization can lead to more complex database designs and queries. It is important to strike a balance between normalization and practicality when designing a database.

First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attributes.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example: Relation EMPLOYEE is not in 1NF because of the multi-valued attribute EMP_PHONE.

EMPLOYEE table

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- In the 2NF, relation must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER table:

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and does not contain any transitive partial dependency.
- 3NF is used to reduce data duplication. It is also used to achieve data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in the third normal form.

A relation is in third normal form if it holds at least one of the following conditions for every non-trivial functional dependency X → Y.

1) X is a super key.
2) Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Example: EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) are transitively dependent on the super key(EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Boyce Codd normal form (BCNF)

- BCNF is the advanced version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE Table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

EMP_ID → EMP_COUNTRY

EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

{ik} INTERVIEW KICKSTART

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Functional dependencies:

EMP_ID → EMP_COUNTRY

EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

- **For the first table:** EMP_ID
- **For the second table**: EMP_DEPT
- **For the third table**: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because the left side part of both the functional dependencies is a key.

Database Security and Authorization

Key Areas of Focus

1. **Data Encryption:** Encrypt data at rest and in transit using SSL/TLS and AES.
2. **Strong Authentication:** Implement multi-factor authentication (MFA) and secure password storage.
3. **Authorization and RBAC:** Define roles and assign permissions accordingly.
4. **Audit Trails and Logging:** Monitor database activity and retain logs for forensic analysis.
5. **Backup and Disaster Recovery:** Regularly backup data and establish recovery plans.
6. **Network Security:** Use firewalls, IDS/IPS, and network monitoring to protect the database.

Additional Learning Resources

1. **Books:**
   - "Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan
   - "Fundamentals of Database Systems" by Ramez Elmasri and Shamkant B. Navathe
2. **Articles and Blogs:**
   - SQL vs NoSQL: What's the Difference?
   - Vector Databases: The Future of AI
3. **Documentation:**
   - PostgreSQL Official Documentation
   - MongoDB Documentation
   - Redis Documentation
4. **Tools:**
   - ERDPlus for Entity-Relationship Diagrams
   - DBDiagram.io for Database Modeling
   - DBeaver for Database Management

Conclusion

Database modeling is essential for creating robust, scalable, and secure databases that serve as the backbone of modern applications.

Understanding relational models, normalization, ACID properties, and security measures ensures that databases are well-designed and optimized for performance. Additionally, staying informed about emerging trends like vector databases and NoSQL technologies allows for more flexible and advanced data management solutions.